

3-1강. Rest API



이영곤



ResponseEntity

- ✓ Spring Framework에서 HTTP 응답 전체(Body + Status + Header)를 제어하기 위한 객체
- ✓ 단순히 데이터를 반환하는 게 아니라, HTTP 레벨까지 명시적으로 제어할 때 사용

구성	설명
Body	실제 응답 데이터 (T)
Status	HTTP 상태 코드 (200, 404 등)
Headers	응답 헤더

Example

```
@GetMapping("/user")
public ResponseEntity<User> getUser() {
    User user = new User("kim");
    return ResponseEntity.ok(user);
}

return ResponseEntity.status(201).body(user);
```

RestTemplate.exchange

- ✓ Spring Framework에서 가장 범용적이고 강력한 HTTP 호출 메서드
- ✓ 단순 GET/POST가 아니라 HTTP 요청 전체(메서드, 헤더, 바디, 응답 타입)를 모두 제어
- ✓ HTTP 요청 생성, 서버 호출, 응답을 `ResponseEntity<T>`로 반환

기본 구조

```
ResponseEntity<T> response =  
    restTemplate.exchange(  
        url,           // 요청 URL  
        HttpMethod.GET, // HTTP 메서드  
        requestEntity, // HttpEntity (헤더 + 바디)  
        responseType   // 응답 타입  
    );
```

Example

```
@GetMapping  
public List<Book> getBooks() {  
    return restTemplate.exchange(  
        baseUrl + "/books",  
        HttpMethod.GET,  
        null,  
        new ParameterizedTypeReference<List<Book>>() {  
        }.getBody();  
    }  
}
```

RestTemplate.exchange

- ✓ header 포함 요청 가능
 - 인증 API 호출시 필수임

Example

```
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(token);

HttpEntity<?> request = new HttpEntity<>(headers);

ResponseEntity<User> response =
    restTemplate.exchange(
        url,
        HttpMethod.GET,
        request,
        User.class
    );
```

RestTemplate.getForObject

- ✓ Spring Framework의 RestTemplate에서 제공하는 가장 단순한 HTTP GET 호출 메서드
 - HTTP GET 전용
 - 응답 Body만 반환 (ResponseEntity 아님)
 - 내부적으로 JSON → 객체 자동 변환

형태

```
T result = restTemplate.getForObject(url, responseType);
```

Example

```
@GetMapping("/{id}")  
public Book getBook(@PathVariable Integer id) {  
    return restTemplate.getForObject(baseUrl + "/books/" + id, Book.class);  
}
```

RestTemplate.postForEntity

- ✓ Spring Framework의 RestTemplate에서 제공하는 HTTP POST 요청을 보내고, 전체 응답(ResponseEntity)을 받는 메서드
 - POST 요청 수행
 - 요청 Body 전송
 - 응답을 ResponseEntity<T>로 반환

형태

```
ResponseEntity<T> response = restTemplate.postForEntity(url, request, responseType);
```

Example

```
@PostMapping
public ResponseEntity<Book> createBook(@RequestBody Book book) {
    return restTemplate.postForEntity(baseUrl + "/books", book, Book.class);
}
```

RestTemplate.delete

- ✓ Spring Framework에서 HTTP DELETE 요청을 보내기 위한 가장 단순한 메서드
 - HTTP DELETE 요청 수행
 - 반환값 없음 (void)
 - 응답 Body를 받지 않음

형태

```
restTemplate.delete(url);
```

Example

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteBook(@PathVariable Integer id) {
    restTemplate.delete(baseUrl + "/books/" + id);
    return ResponseEntity.noContent().build();
}
```

- ✓ 클라이언트에서 REST API 호출시 사용
- ✓ 페이지 전체를 새로 고침하지 않고 비동기로 서버와 통신
 - 비동기 요청
 - 페이지 reload 없음
 - 서버에서 JSON 받아서 화면 갱신

형태

```
const res = await fetch(API);
```

Example

```
async function loadBooks() {  
  try {  
    const res = await fetch(API);  
    if (!res.ok) throw new Error(`서버 오류 (${res.status})`);  
    const books = await res.json();  
    renderBooks(books);  
  } catch (e) {  
  }  
}
```


Example

```
let res;  
if (id) {  
  res = await fetch(`${API}/${id}`, {  
    method: 'PUT',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify(book)  
  });  
} else {  
  res = await fetch(API, {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify(book)  
  });  
}
```

id 존재 여부에 따라 POST vs PUT을 분기

서버에 json 형태로 데이터 보냄을 명시

JS 객체 → JSON 문자열 변환

- ✓ Spring Cloud OpenFeign 기반의 선언형 HTTP 클라이언트
- ✓ 인터페이스만 정의하면 REST 호출 코드를 자동으로 만들어 줌
 - 인터페이스만 정의
 - 구현 코드 없음
 - 자동으로 HTTP 호출 생성

동작 방식

- ✓ Spring이 @FeignClient 스캔
- ✓ 런타임에 Proxy 객체 생성
- ✓ 메서드 호출 → HTTP 요청으로 변환
 - HTTP 요청 생성, JSON 직렬화/역직렬화, 에러 처리

Example

```
@FeignClient(name = "book-service")
public interface BookClient {

    @GetMapping("/books")
    List<Book> getBooks();

    @PostMapping("/books")
    Book createBook(@RequestBody Book book);
}
```

인터페이스

```
@Service
public class BookService {
    private final BookClient bookClient;
    public BookService(BookClient bookClient) {
        this.bookClient = bookClient;
    }
    public List<Book> getBooks() {
        return bookClient.getBooks();
    }
}
```

페인클라이언트 활용